Trabajo Práctico: Motor de búsqueda de funciones similares vía AST



Materia: Lenguajes Formales y Autómatas
Profesores: Ferrero Mariano, Rossanigo Ariel
Alumnos: Cavallo Tobias, Clemenz Julian
Fecha entrega: 30/10/2026

Desarrollamos una herramienta con el lenguaje de programación Python, la cuál realiza la tarea de obtener funciones de un determinado archivo y evalúa a través de la estructura de las mismas cuan similares son, mostrando un puntaje. Para realizar esta herramienta tomamos como base herramientas ya desarrolladas de detección de código duplicado como MOSS, JPlag, etc.
La idea central de dicha herramienta es comparar funciones por su estructura y no por su texto. Para desarrollar esto se tuvo en cuenta lo que vimos en la materia cuando se mencionó que un compilador primero separa el código en palabras(análisis léxico) y después revisa que respeten las gramáticas, armando un árbol de derivación(análisis sintáctico). Python realiza toda esta operación con la función ast.parse(), y nosotros trabajamos sobre ese árbol, que en este lenguaje se llama AST. Si un archivo tiene errores de sintaxis, no se puede armar dicho árbol, así que se descarta.

Diseño

Para el desarrollo de esta herramienta implementamos 4 módulos:
extraccion.py → normalizacion.py → similitud.py → buscar_similares.py

A continuación vamos a desarrollar brevemente la función de cada uno.

Extracción: Recorre la carpeta indicada y todas sus subcarpetas, parseando cada archivo con ast.parse() para traducirlo a un árbol (AST). Usamos un **ast.NodeVisitor** con un método por tipo de nodo (**visit_FunctionDef** y **visit_AsyncFunctionDef**), de forma parecida a un parser recursivo descendente, que tiene una rutina por no terminal. Además de las funciones comunes, detecta funciones async, métodos de clases y funciones anidadas; en los ejemplos incluimos un caso de cada una para probarlo. Si un archivo tiene errores de sintaxis, el programa avisa y sigue con los demás. De cada función guardamos su nombre, archivo, línea y subárbol.

Normalización: Sobre una copia del árbol, para no modificar el original aplicamos un **ast.NodeTransformer** el cual se encarga de:

- Reemplazar variables y parámetros por VAR1, VAR2…, en orden en el que aparecen en el recorrido. Para cada función utilizamos un diccionario para ir manteniendo un recuerdo de qué “VAR” tiene que variable y funciona como una tabla.
- Reemplaza todos los numero, textos, true, false, None por “LIT”.
- Reemplaza el nombre de la función por “FUNC”.
- Elimina los comentarios, para que una función documentada y otra sin documentar no se consideran distintas.
- **No** normaliza las funciones propias de Python (len, print, range...), porqué usar una u otra es una diferencia real en la lógica.

Serializacion: Convertimos el arbol normalizado en una lista de tokens con un recorrido pre-orden implementado con una funcion recursiva la cual, primero se anota el nodo y despues se recorren sus hijos en orden. Este orden coincide con el de derivacion left-most. Por cada nodo anotamos su tipo (For, Assign…) y su valor normalizado si tiene (VAR1, LIT). Omitimos los nodos Load y Store, porque agregan ruido apareciendo en cada variable. Si incluimos los operadores Add, Sub…, porque distinguen operaciones diferentes.

Métrica y umbral:

Usamos **similitud de Jaccard sobre n-gramas**: dividimos cada lista de tokens en pedazos de *n* tokens seguidos y calculamos la proporción de pedazos en común sobre el total. El resultado va de 0 (nada en común) a 1 (iguales). La elegimos porque no depende de la posición de cada pedazo y, por eso, tolera las líneas que en algunos casos están reordenadas. Levenshtein, en cambio, compara posición por posición y penaliza mucho mover una línea.

Elección de n: probamos n de 1 a 5 y medimos el **margen** entre el clon con menor puntaje y el no clon con mayor puntaje. Descartamos n=1 porque ignora el orden de los tokens. De los demás, **n=2** dio el mayor margen (0.73 contra 0.65). Con n≥4, los clones dejan de distinguirse.

Umbral: 0.69, el punto medio entre esos dos valores. Desde ese puntaje, dos funciones se consideran candidatas a duplicado.

Pruebas

Implementamos 5 test con pytest, 4 obligatorios y 1 agregado, todos aprobados:

- **Positivos**: clon con variables renombradas (puntaje 1.00) y con líneas reordenadas (0.73). Ambos superan el umbral.
- **Negativos**: sumar_elementos contra invertir_cadena (0.65), que son las funciones distintas más parecidas entre sí, y buscar_maximo contra contar_vocales (0.42). Ninguno llega al umbral.
- **Programa completo**: test agregado, buscar las funciones similares a sumar_elementos, la primera es su clon calcular_total.

Limitaciones

- Clones con otra estructura: no detecta funciones que hacen lo mismo con código distinto, por ejemplo un **for** contra un **while.**
- Reordenamiento: al cambiar líneas de lugar también cambia la numeración de los VAR, por lo que el puntaje baja aunque la función sea la misma. Con muchos cambios, un clon podría quedar debajo del umbral.
- Funciones muy cortas**:** generan pocos tokens, y una diferencia mínima cambia mucho el puntaje
- Calibración con pocos ejemplos.

Posibles mejoras

- Calibrar n y el umbral con una cantidad mayor de ejemplos.
- Enumerar las variables de una forma que no dependa del orden de las líneas.
- Comparar los árboles directamente en lugar de las listas.
- En caso de tener un repositorio grande, evitar la comparación todos contra todos, agrupando funciones bajo algun criterio, como por ejemplo, tamaño.